

SSI Implementation and Validation: Last Few Days Summary

From core algorithm to a full Forward-Model-to-SSI validation ladder

Summary for supervision discussion

This note summarises everything built and learned over the last few days of Stochastic Subspace Identification (SSI) implementation: the core algorithm files, the unit and integration tests used to validate them, and a multi-level validation ladder connecting SSI to the already-validated hydroelastic Forward Model. Findings are marked in green; genuinely open items are marked in red.

1 Implementation status: all files built

File	Purpose	Validation status
<code>buildHankelMatrix.m</code>	Past/future block Hankel construction	Exact match, hand-computed case
<code>hankelProjection.m</code>	QR-based projection $P = Y_f / Y_p^{\text{ref}}$	Exact match vs. direct formula
<code>extractStateSpace.m</code>	SVD extraction of (A, C) at order n	Known system + Forward Model twin test
<code>modalParameters.m</code>	Eigenvalue $\rightarrow (\omega, \zeta, \Phi)$	Branch-cut bug found and fixed (Section 3.1)
<code>computeMAC.m</code>	Modal Assurance Criterion	Third-party squared-numerator bug ruled out
<code>sweepModelOrders.m</code>	Per-order extraction, conjugate dedup	Multi-mode system, hard invariant check
<code>classifyPoleStability.m</code>	Order-to-order matching, class 0–3	Full hierarchy, one-to-one assignment
<code>buildModalBranches.m</code>	Reconstructs order-tracked pole branches	Known structure + integration test
<code>computeBranchPersistence.m</code>	Sliding-window persistence criterion	Validated; parameters shown context-dependent
<code>syntheticSensorData.m</code>	Bridges Forward Model output to multi-channel SSI input	Exact regression against direct calls, feeds cleanly into Hankel construction

Architecture. No single function decides whether an identified pole is physically real:

Hankel $\rightarrow$ projection $\rightarrow$ state space at order $n \rightarrow$ modal parameters
 $\rightarrow$ pairwise classification $\rightarrow$ branches $\rightarrow$ persistence.

Persistence answers only “is this a numerically consistent candidate”; physical interpretation is deliberately left for later.

2 Background: how SSI works, and key concepts used throughout

This section explains the SSI algorithm itself and the concepts (MAC, cross-MAC, branches, persistence) referenced throughout the rest of this note, so the results below can be read without needing the surrounding implementation discussion.

2.1 The problem SSI solves

A single sensor (or several) records a time series believed to come from some underlying linear dynamical system: an ice floe responding to wave forcing, for instance. Nothing about the internal dynamics of that system is known in advance, not even how many meaningful internal states it has. SSI’s job is to look only at the recorded output and reconstruct a discrete-time state-space model

$$x_{k+1} = Ax_k + w_k, \quad y_k = Cx_k + v_k,$$

where x_k is an unmeasured internal state, y_k is what the sensors actually recorded, A describes how the system evolves from one instant to the next, and C describes how the internal state shows up in the measurements. w_k, v_k are noise terms. Crucially, no input/forcing term appears: SSI is an *output-only* method, appropriate exactly because the real wave forcing on a floe is never directly measured.

2.2 From raw data to a candidate model, step by step

Step 1: the Hankel matrix (`buildHankelMatrix.m`). The recorded time series is reorganised into two blocks: a “past” block Y_p , containing short windows of recent history, and a “future” block Y_f , containing what happened immediately after each of those windows. Each *column* of these matrices is one such window, taken at a different starting point in the recording, so that the same sliding window is repeated across the whole record as separate data points.

Step 2: the projection (hankelProjection.m). SSI asks: of everything in the future block, how much is actually predictable from the past block alone? Projecting Y_f onto the row space of Y_p answers exactly this, producing a matrix P that represents “the predictable part of the future given the past”. This is computed via a QR factorisation for numerical robustness, rather than a more direct but numerically worse formula.

Step 3: singular value decomposition and model order (extractStateSpace.m). A foundational theorem of subspace identification states that this projection factors exactly as $P = O_i \hat{X}_i$, where O_i is the *extended observability matrix* $[C; CA; CA^2; \dots]$ and $\hat{X}_i$ is a sequence of state estimates. Neither factor is directly available, only their product; the singular value decomposition $P = U\Sigma V^T$ recovers them. Truncating to the largest n singular values selects a *model order*: how many internal states the reconstructed system is assumed to have. This order is not known in advance and must be swept over a range (see Section 2.3), rather than fixed at one value.

Step 4: recovering A and C . C is simply the first block row of O_i . A is recovered from a structural property of O_i itself: dropping its last block row and dropping its first block row give two matrices related by exactly one multiplication by A , so A follows from a single least-squares solve, without needing any further data.

Step 5: modal parameters (modalParameters.m). A ’s eigenvalues λ (generally complex) are converted to continuous-time poles via $\lambda_c = \ln(\lambda)/\Delta t$. A genuine oscillatory mode appears as a complex-conjugate pair $\lambda_c, \lambda_c^* = -\zeta\omega \pm i\omega\sqrt{1-\zeta^2}$, directly giving the natural frequency $\omega = |\lambda_c|$ and damping ratio $\zeta = -\text{Re}(\lambda_c)/|\lambda_c|$ of that mode. The corresponding *mode shape* (more precisely, for a forced signal, the spatial response pattern) is $\Phi = C\Psi$, where Ψ holds A ’s eigenvectors.

2.3 Why the model order must be swept, not assumed

There is no way to know the correct number of internal states in advance, and real (noisy) data rarely shows a clean gap in the singular values that would reveal it directly. `sweepModelOrders.m` instead repeats Steps 3–5 at every order from 1 up to some $n_{\max}$, producing a separate candidate set of frequencies, damping ratios, and mode shapes at each order. Genuine physical content tends to reappear consistently across many orders; noise and over-parameterisation artefacts tend not to. Distinguishing the two is the job of the machinery described next.

One subtlety handled internally: every genuine oscillatory pole appears twice in A ’s eigenvalues (a complex-conjugate pair), so `sweepModelOrders.m` keeps only one representative of each pair, otherwise every real mode would be silently double-counted.

2.4 The Modal Assurance Criterion (MAC) and cross-MAC

Comparing two mode shapes (or spatial response vectors) numerically is complicated by the fact that output-only methods can never pin down an absolute scale or phase: a shape vector ϕ and $5e^{i0.3}\phi$ represent exactly the same underlying pattern. The Modal Assurance Criterion is built to be blind to this:

$$\text{MAC}(\phi_a, \phi_b) = \frac{|\phi_a^H \phi_b|^2}{(\phi_a^H \phi_a)(\phi_b^H \phi_b)} \in [0, 1],$$

where $(\cdot)^H$ is the conjugate transpose. $\text{MAC} = 1$ means the two shapes are identical up to scale and phase; $\text{MAC} = 0$ means they are orthogonal, sharing no common spatial pattern at all. Crucially, with only a single sensor channel, MAC between any two nonzero scalars is *always exactly* 1, by the formula above; MAC only becomes meaningful once at least two channels are available to actually distinguish different spatial patterns.

MAC is used in this project in two distinct ways, both appearing in the results below:

- **Stability MAC** (inside `classifyPoleStability.m`): compares a candidate pole’s shape at one model order against its own shape at the previous order, as one ingredient (alongside frequency and damping) in deciding whether that pole is being tracked consistently as order increases.
- **Matching / cross-MAC** (in the Forward-Model validation ladder, Sections 5–8): compares an SSI-identified shape against the *known, true* Forward Model spatial response. “Matching MAC” compares a shape against the response it is supposed to represent (should be high); “cross-MAC” compares it against a *different* frequency’s known response, deliberately testing whether the method can tell the two apart (should be low, or more precisely: should match whatever the true Forward-Model-only cross-MAC actually is, since that is the real ceiling on achievable discrimination, not a fixed number chosen in advance).

2.5 Branches: tracking one pole across many orders

Sweeping model order produces an independent list of candidate poles at every order, with no inherent link between, say, the third pole at order 6 and the third pole at order 7. `classifyPoleStability.m` builds that link explicitly: for every pole at order n , it searches for the best matching pole at order $n-1$ (gated first by frequency similarity, then scored by damping and MAC), assigning matches one-to-one so that no single pole at one order is claimed by two different poles at the next. Each match is labelled with a class:

Class	Criterion
0	No compatible match at the previous order (frequency gate failed)
1	Frequency matched, damping did not
2	Frequency and damping matched, MAC did not
3	Frequency, damping, and MAC all matched (“fully stable”)

`buildModalBranches.m` then threads these order-to-order links together into full *branches*: a branch is simply a pole followed across as many consecutive orders as it keeps matching. This is deliberately not called a “continuous” or gap-free trace: if a pole fails to find any match at some order, its branch ends there, and if a similar pole reappears at a later order, that starts an entirely new branch, with no memory of the earlier one. This gap-intolerant design is what led directly to the branch-fragmentation finding in Section 4.3.

2.6 Persistence: deciding which branches are trustworthy candidates

A branch that is class-3 stable for every single order it appears at is obviously a strong candidate. Real (noisy) data rarely looks this clean: a genuinely real mode might show class-3 links at most orders but drop to class 1 or 2 at an isolated order in between. Requiring an unbroken run of class-3 links throughout is too strict, and was shown empirically (Section 4.2) to under-report genuine, repeatedly-stable modes. `computeBranchPersistence.m` instead uses a sliding-window criterion: a branch is declared persistent if *at least one* window of W consecutive order-transitions contains at least k class-3 links (a “ k of W ” rule), rather than requiring all W of them. This explicitly does not claim the branch is stable for its entire lifetime, only that convincing evidence of stability exists somewhere in its history: a deliberate, narrower definition suited to flagging trustworthy candidates for further (later, physics-based) interpretation, not a claim about physical reality on its own.

2.7 What persistence does, and does not, establish

It is worth stating explicitly, since it recurs throughout this document: a persistent branch means SSI has found a *numerically consistent candidate* across model order. It does not, on its own, mean that candidate is a genuine floe vibration mode. The same machinery, applied to a forced but undamped tone (Section 6), or to a purely mathematical artefact of over-parameterisation (Section 4.3), can produce an equally persistent-looking branch. Separating “numerically consistent” from “physically meaningful” is deliberately left to a later interpretive stage, outside the scope of the functions summarised in Section 1.

3 Phase 1: core algorithm validation

3.1 A real bug caught by a structural invariant

Branch-cut artifact in `modalParameters.m`. A real, negative discrete eigenvalue produces a spurious $+i\pi/\Delta t$ term after `log()` (MATLAB’s complex branch cut), indistinguishable from a genuine oscillatory pole with no conjugate partner. Caught because `sweepModelOrders.m`’s tester asserted a hard bound (a real system of order n has at most $\lfloor n/2 \rfloor$ conjugate pairs); the phantom pole violated it at every odd order. Fixed by testing `imag(lambda)` (before the branch cut) rather than `imag(lambda_c)` (after it).

3.2 Pass 1 integration test: noise-free vs. noisy

A synthetic 3-mode system ($\omega = 1.2, 2.7, 4.1$ rad/s, single channel, true state order 6) was swept from model order 1 to 14.

Noise-free: exact recovery. All three modes recovered exactly from order 6 onward, with an unbroken class-3 run through order 14.

Noisy (1% of signal std): degradation, not failure. All three modes still showed repeated class-3 links, just not consecutively. This motivated `computeBranchPersistence.m`’s sliding-window design (“ k of W ” rather than strict consecutiveness).

3.3 Branch fragmentation and its diagnosis

Applying persistence directly to the noisy data initially showed no persistent branches, for a subtler reason than weak stability: `buildModalBranches.m` does not bridge gaps, so genuine modes were fragmenting into branches too short to satisfy any window size.

Diagnosis via a controlled `freqTo1` sweep. Raw per-order frequency estimates were inspected directly rather than adjusting any downstream function. Two of three modes had a compatible pole at every order, drifting more than the 1% gate allowed between some orders; widening to 5% reconnected these into single, unbroken, fully class-3 branches. The third mode (4.1 rad/s) had no compatible pole at any tolerance, at

any order: not a gating problem, an apparent estimation failure under this noise realisation.

Open: the missing 4.1 rad/s mode. Not yet explained (weaker modal weighting, noise realisation, record length, and numerical conditioning are all candidate explanations). Requires testing across multiple noise seeds.

3.4 MAC triviality at $l = 1$, and its consequence

MAC provides zero discrimination with one sensor channel. For any two nonzero scalars a, b , $\text{MAC}(a, b) = 1$ identically. Every single-channel test run so far had therefore used frequency and damping matching only.

Experiment: two close hand-built modes (2.70, 2.80 rad/s, 3.7% separation). With one channel, $\text{freqTol} = 5\%$ produced branches that visibly oscillated between both true frequencies: a genuine wrongful merge. With a second channel carrying different modal weighting, the identical tolerance cleanly separated the two modes, each locking onto its own true frequency for eleven consecutive orders.

Consequence: freqTol is not a single universal constant. The appropriate value depends on sensor count and how distinct their modal sensitivity is, not a value fixed in advance.

This was direct, independent software-level evidence for the multiple-sensor decision already made on other grounds (identifiability of D , R , E^v).

4 Phase 2: connecting SSI to the Forward Model

4.1 The bridge: `syntheticSensorData.m`

Every prior test used hand-built oscillators. `syntheticSensorData.m` calls the already-validated `evaluateSpectralDeflection.m` once per sensor location and stacks the results as rows, producing exactly the $l \times n$ matrix `buildHankelMatrix.m` expects. Validated by exact regression against direct calls (single- and multi-sensor), confirmation that distinct sensor locations give distinct signals, and a clean handoff into `buildHankelMatrix.m`.

4.2 Level 1: single monochromatic Forward Model signal

Tests whether SSI recovers a known incident frequency from genuine, spatially-distributed hydroelastic response (4 sensors: centre, mid-radius, two near-edge points), rather than from a synthetic oscillator.

Interpretive caveat, kept explicit throughout. `evaluateSpectralDeflection.m` produces a forced steady-state response, not a free-vibration record. A pure tone has, mathematically, an exact minimal realisation with unit-circle eigenvalues ($\zeta = 0$ exactly). Recovering ω here confirms SSI can extract a known frequency from real spatial floe physics; it does *not* confirm ω is a natural hydroelastic mode.

Result: essentially exact recovery at every order. At $\omega = 5.5$ rad/s, frequency error was $\sim 10^{-15}$ (machine precision) at every model order 2 through 10, and identified damping sat at $\sim 10^{-14}$ to 10^{-16} , exactly as predicted for a forced tone. A negative control (testing the same recovered result against a deliberately wrong target, 5.0 rad/s) correctly reported a 10% mismatch and failed as it should, confirming the test is not vacuous.

4.3 Level 2: two close frequencies, formal spatial identification

Two components (2.70, 2.80 rad/s, the same stress case as the earlier hand-built experiment) were built by hand into a `specData`-shaped struct (bypassing JONSWAP's automatic binning) so exact frequencies, amplitudes, and phases were known precisely.

Forward-Model-only cross-MAC, computed before any SSI involvement. The true spatial responses at these two close, low frequencies gave $\text{MAC}_{\text{FM}, \text{cross}} = 0.9995$: essentially indistinguishable, independent of any identification method. SSI's own cross-MAC matched this to four decimal places. Both branches recovered their exact true frequencies at every order 4 through 12, and matching MAC (SSI shape vs. its own true response) was 1.0000 throughout. The initial pass/fail design (requiring cross-MAC < 0.5) was revised: the correct success criterion is $\text{MAC}_{\text{SSI}, \text{cross}} \approx \text{MAC}_{\text{FM}, \text{cross}}$, i.e. SSI faithfully reproducing whatever the true physical similarity actually is, not an arbitrary threshold.

4.4 Level 2A: frequency-regime study

Sensor locations held fixed; $\omega_1 = 2.70$ fixed, ω_2 varied.

ω_1 (rad/s)	ω_2 (rad/s)	$\text{MAC}_{\text{FM,cross}}$	$\text{MAC}_{\text{SSI,cross}}$	Agree
2.70	2.80	0.9995	0.9995	yes
2.70	4.10	0.8263	0.8263	yes
2.70	6.00	0.1314	0.1314	yes
2.70	8.00	0.1886	0.1886	yes
2.70	9.50	0.3585	0.3585	yes

Spatial distinguishability is strongly frequency-dependent, not simply frequency-separation-dependent. The sequence is non-monotonic (a minimum near $\omega_2 = 6.00$), so no claim is made that higher frequency or larger separation alone improves distinguishability. SSI reproduced the true Forward Model cross-MAC to four decimal places at every single pair tested, from near-total similarity to genuine separability. This demonstrates SSI preserves the actual spatial information content of the response, not only its dominant frequency.

Scope note. Only ω_1 was held fixed while ω_2 varied; frequency separation and absolute frequency regime were not varied independently, so their effects remain conflated in this dataset.

4.5 Level 2B: sensor-placement study

Frequencies fixed at the genuinely distinguishable pair identified in Level 2A (2.70, 6.00 rad/s, $\text{MAC}_{\text{FM,cross}} = 0.1314$); only sensor locations varied across four configurations (all 4 sensors, matching the agreed sensor count).

Configuration	$\text{MAC}_{\text{FM,cross}}$	$\text{MAC}_{\text{SSI,cross}}$	Agree
Original (centre, mid-radius, 2 near-edge)	0.1314	0.1314	yes
Edge, 4 quadrants	0.1881	0.1881	yes
Radial spread + 1 off-axis	0.8613	0.8613	yes
Edge, narrow θ spread	0.2696	0.2696	yes

Sensor placement matters enormously, and not in the naively expected direction. The radial-spread configuration performed far worse (MAC 0.86) than every edge-clustered configuration (0.13–0.27), directly contradicting a naive “more radial coverage is better” assumption. Matching MAC was 1.0000 for both frequencies in every configuration, and cross-MAC agreement between SSI and the known Forward Model response held exactly, at every common model order (4 through 12, verified explicitly, not just at the final order), for all four configurations.

Scope note. This result is specific to the 2.70/6.00 rad/s pair and the four tested configurations. The non-monotonic pattern among the three edge-based configurations (4-quadrant spread performing worse than the narrower baseline) is itself evidence against generalising “more angular spread is better” from this one case. Not yet a general sensor-placement principle.

5 Overall conclusions

What is now firmly established. Across every test in the validation ladder (Level 1 through 2B), spanning a single tone, close and widely-separated frequency pairs, near-identical and clearly-distinct true spatial responses, and four different sensor geometries, SSI recovered the correct frequency, the correct spatial response (matching $\text{MAC} = 1.0000$ throughout), and the correct cross-frequency spatial similarity (exact agreement with the Forward-Model-only ground truth) in every single case. At no point has SSI’s own machinery been the limiting factor; every constraint found (indistinguishable close frequencies at low α , radial placement hurting distinguishability for one tested mode pair) has been a property of the physics or sensor geometry, and SSI reported it faithfully.

What remains open or context-specific.

1. The missing 4.1 rad/s mode in the noisy 3-mode system (Section 2.3): unresolved, needs multi-seed testing.
2. `freqTo1` has no single validated default; it is confirmed to depend on sensor count and configuration.
3. The Level 2B sensor-placement finding is a single data point for one frequency pair, not a general design rule.

4. No noise has been added to any Level 1/2/2A/2B test; all Forward Model validation so far is noise-free. Robustness under realistic noise, for genuine multi-sensor physics, has not yet been tested.

6 Roadmap: proposed next steps

The validation ladder so far (Level 1 through 2B) has been entirely deterministic and noise-free. A further roadmap has been proposed, extending it toward genuinely realistic conditions before any real buoy data is involved. This section records that roadmap, together with amendments made to it and the reasoning for the next concrete step.

6.1 The proposed levels

Level	Name	Question
2C	Systematic sensor-placement search	Can placement be chosen to maximise spatial distinguishability, beyond the four hand-picked configurations already tested?
3	Frequency proximity and amplitude imbalance	How close can two components become, and how unequal in amplitude, before SSI can no longer reliably separate them?
4	Phase variation	Does recovery remain robust once phases are no longer zero, as used throughout Levels 1–2B?
5	Additive noise, multiple SNRs and seeds	How does frequency bias, frequency spread, branch persistence, and matching/cross-MAC degrade statistically as noise increases?
6	Sensor imperfections	Gain errors, bias, a failed sensor, timing perturbation: does the method survive realistic instrumentation flaws?
7	Broadband/realistic wave spectra	Two tones → several tones → a full spectrum: does fidelity carry through to the eventual JONSWAP-forced case?
8	Known free-vibration response	Given a signal with KNOWN natural frequencies and damping (not a forced response), does SSI recover them?

6.2 An important standing caveat on the whole ladder

Levels 1 through 7 validate forced-response fidelity, not natural mode recovery. This distinction was stated explicitly as early as Level 1 (Section 6) and applies to every level above except Level 8: each one asks whether SSI faithfully recovers whatever a FORCED spectral input produces, at increasing levels of realism. None of them, however many pass cleanly, demonstrate that SSI can find the floe’s own natural hydroelastic modes from a free-vibration response. Level 8 is the only level that changes what kind of claim can be made about the floe itself, rather than about SSI’s fidelity to a forcing signal. Its priority should be reconsidered relative to the other levels once project time constraints are clearer, rather than treated only as a final item to reach eventually.

6.3 Two amendments made to Level 2C and 3

- **Level 2C needs an explicit choice of goal before it is built.** Generating many (e.g. 20–50) candidate configurations and ranking their Forward-Model cross-MAC characterises the landscape of possible outcomes, but does not by itself constitute an *optimisation*. If the intent is genuinely to search for a good configuration, a real search procedure (e.g. coordinate or gradient-free search over configuration space) is needed; if the intent is only to describe how outcomes vary across many configurations, that is a more modest and equally legitimate goal, but should be named as such (a placement *search or survey*, not a claim of *optimal* placement) to avoid overclaiming, exactly as the Level 2B scope note already does for the four hand-picked configurations.
- **Level 3 is proposed to be attempted before Level 2C.** Level 3 tests a property of SSI’s own resolution using the existing baseline sensor configuration, with no new supporting infrastructure required. Level 2C requires building a configuration generator and a ranking or search procedure before it produces any result at all. Given how inexpensively Level 2A’s finding was obtained (five frequency pairs, no new infrastructure), Level 3 is expected to be similarly cost-effective and is recommended first, with 2C following once its own scope (search vs. survey) is settled.

6.4 Connection back to still-open Phase 1 items

Phase 1’s open items should be revisited within Level 5, not separately. The missing 4.1 rad/s mode and the context-dependence of `freqTo1` (Section 4) were both found using a hand-built oscillator under additive noise. Level 5’s systematic noise study, once built, is the natural place to re-examine both questions using genuine multi-sensor Forward Model data instead, rather than treating them as a separate, disconnected investigation.

6.5 Idea for immediate next step

Level 3 (frequency proximity and amplitude imbalance). It directly extends the already-established 2.70/2.80 rad/s stress case (Section 7) toward the actual question that case was chosen to probe: not merely whether two close frequencies can be separated at all, but how close they can become before separation fails. It requires no new infrastructure beyond what already exists, and is expected to produce a result of comparable clarity and cost to Level 2A's frequency-regime sweep.